Giraffle Project

AGENT LOOP - MCP - LANGGRAPH INTEGRATION

YAHYA EFE KURUÇAY - 20220808005 - CSE 336 Advanced Web Programming

Note: You can go directly to the GitHub link via the code paths.

1. Agent Loop (Spotter'ın Beyni)

My project includes an AI assistant called "**Spotter**". The `src/domain/agent/loop.ts` file determines how this assistant thinks in the background while talking to us.

Here's a standard and very efficient **Tool Calling** loop. (I examined how they implemented it from the source code of the **Claude Code** tool and integrated a similar one into my system.) Using the streamText function of the **Vercel AI SDK**, we operate with the following logic:

1. **Provide context:** We tell the Spotter, "You are an assistant, the user has these folders and these notes" **(buildSystemPrompt).**
2. **Provide Tools:** We tell the Spotter, "If you want to search for notes, use this tool; if you want to search the internet, use this tool." (`buildAgentToolset`).
3. **Start Loop:** The Spotter takes your problem, runs a tool if necessary (e.g., searches for notes in the database), looks at the returned response, and repeats this process until the loop completes (or reaches the maximum number of steps DEFAULT_MAX_STEPS = 12).

How did we implement it?

```
// src/domain/agent/loop.ts (Basitleştirilmiş)
const result = streamText({
  model: provider("gpt-4o-mini"),
  system: systemPrompt, // "Sen Spotter'sın, notlara erişebilirsin..."
  messages: modelMessages,
  tools: toolset.tools, // Not arama, klasör açma, web araması vb. araçlar
  maxSteps: 12, // Yapay zeka kendi kendine en fazla 12 kez tool çağırabilir

  // Her adım bittiğinde tool (araç) kullanımını logla
  onStepFinish: async (step) => {
    for (const call of step.toolCalls ?? []) {
      await recordToolAudit({ toolName: call.toolName, status: "success" /* ... */ });
    }
  }
});
```

A small detail: We did something very clever and wrote a function called compactMessages (borrowed from the Claude code implementation :)). We trim old tool responses so that the context window doesn't get bloated when the AI calls too many tools. This is a great tactic for saving tokens. The implementation:

```ts
 */
export function compactMessages(
  messages: ModelMessage[],
  options: CompactionOptions = DEFAULT_COMPACTION,
): { messages: ModelMessage[]; compacted: boolean; before: number; after: number } {
  if (messages.length <= options.triggerCount) {
    return { messages, compacted: false, before: messages.length, after: messages.length };
  }
  const head = messages.slice(0, 1);
  const tail = messages.slice(-options.keepTail);
  const middle = messages.slice(1, -options.keepTail);
  const compactedMiddle = middle.map((msg) => {
    if (msg.role !== "tool") return msg;
    if (!Array.isArray(msg.content)) return msg;
    const blocks = msg.content as ToolContentBlock[];
    return {
      ...msg,
      content: summariseToolContent(blocks, options.perResultCharLimit),
    } as ModelMessage;
  });
  const next = [...head, ...compactedMiddle, ...tail];
  return { messages: next, compacted: true, before: messages.length, after: next.length };
}

```

---

2. LangGraph Integration (Inbox Triage / Human-Verified Transactions)

The standard Agent Loop (above) is great for instant chats, but it falls apart when it comes to things like, "**Read 20 notes in my inbox, organize them into folders, present me with a plan, and move them if I approve.**" Because freezing the process in the instant loop and waiting for human approval is difficult.

This is where **LangGraph** comes in (src/domain/agents/inbox-triage/graph.ts). With LangGraph, we defined the process as a "state machine".

The logic goes like this:

1. Load notes and folders from the database (loadWorkspace).
2. Pull notes from the inbox (loadInboxNotes).
3. Request a plan (JSON) from the AI on how to organize them (proposeActions).
4. **STOP** (Interrupt): This is the magic of LangGraph! You freeze the process in the approvalNode. The state is saved to the Postgres database.
5. When you say "I approve this move, I reject this one" via the UI, LangGraph wakes up (resumeInboxTriageRun) and only applies the ones you approved (applyApprovedActions).

```typescript
// src/domain/agents/inbox-triage/graph.ts (Aksiyonların çizildiği yer)
function buildInboxTriageGraph() {
  return new StateGraph(InboxTriageAnnotation)
    .addNode("loadWorkspace", loadWorkspaceNode)
    .addNode("proposeActions", proposeActionsNode)
    .addNode("approval", approvalNode) // <-- İnsan onayının beklendiği durak
    .addNode("applyApprovedActions", applyApprovedActionsNode)
    // Bağlantıları (Edge) çiziyoruz:
    .addEdge(START, "loadWorkspace")
    .addEdge("loadWorkspace", "proposeActions")
    .addEdge("proposeActions", "approval") // Plandan sonra onaya git
    .addEdge("approval", "applyApprovedActions") // Onay gelirse uygula
    .addEdge("applyApprovedActions", END);
}
```

---

3. MCP (Model Context Protocol) Integration

In project project, the MCP **operates bi-directionally.**

A. Giraffle operates as an MCP Server:

We are opening up our internal tools for reading and writing notes (notes_search, notes_create) to the outside world. So, if we give Claude Desktop the URL and Token of Giraffle, Claude Desktop can read and write notes from our Giraffle database.

```typescript
// src/mcp/server.ts
export function createGiraffleMcpServer() {
  const server = new McpServer({ name: "giraffle-mcp", version: "0.2.0" });

  // Sistemindeki tüm tool'ları MCP üzerinden dışarı açıyorsun
  for (const def of INTERNAL_TOOL_DEFINITIONS) {
    server.registerTool(def.name, def.inputSchema, async (input, extra) => {
      // Dışarıdan gelen isteği çalıştırıp veritabanına yazar/okur
      const output = await def.execute(input, { userId });
      return { content: [{ type: "text", text: "İşlem başarılı" }] };
    });
  }
  return server;
}
```

B. Giraffle is operating as an MCP Client:

Our system (Spotter) can connect to custom MCP servers written by others outside the system [(src/domain/agent/mcp-clients.ts).]

For example, we enter the URL of a GitHub MCP Server, WebSearch MCP servers, or a private MCP server in the company into Giraffle's settings. Spotter connects to that server, discovers the tools there, and can use those tools instantly while talking.

```
// src/domain/agent/mcp-clients.ts (Basitleştirilmiş)
// Dış dünyadaki bir MCP'ye bağlanma
const client = await createMCPClient({
  transport: { type: "http", url: "https://disaridaki-bir-sunucu.com/mcp" }
});
const tools = await client.tools(); // Dışarıdaki toolları öğren ve Spotter'ın beynine ekle!
```

In short, what have I created?

1. The chat interface (Spotter) is powered by the fast and cost-effective Vercel AI SDK.
2. Background operations (Inbox Management) are powered by LangGraph, which can be interrupted and requires human approval.
3. Thanks to ecosystem expansion **(MCP),** Giraffle is not a static application; it can be managed externally and incorporate external resources into its intelligence.

It's a self-hosted project and will remain so. In today's world, where platform dependency is high and constantly increasing, I believe platform-agnostic structures are becoming increasingly important.

Yahya Efe Kuruçay
efekurucay.com
github.com/efekurucay